

7,75

Questão 1 [2 pontos]:

O Centro Acadêmico do IC/UFAL fará uma eleição com d candidatos. Um total de n alunos estão aptos a votar. Dado um vetor $A[1..n]$ contendo valores inteiros $\{1, \dots, d\}$, implemente um algoritmo que decida se houve uma maioria de votos ou não. Prove a corretude de seu algoritmo.

Questão 2 [2 pontos]:

Considere o seguinte algoritmo:

```
1: procedure ALGORITMO X(vetor A[1...n], inicio, fim)
2:   if inicio = fim then
3:     return A[inicio]
4:   end if
5:   meio ← inicio + (fim - inicio)/2
6:   a ← X(A, inicio, meio)
7:   b ← X(A, meio + 1, fim)
8:   if a < b then
9:     return b
10:  else
11:    return a
12:  end if
13: end procedure
```

Explique o que ele faz e prove sua corretude.

Questão 3 [1 ponto]:

Seja $f(n)$ uma função dos inteiros nos reais. Justifique:

1. Qual a definição formal de $O(f(n))$?
2. É verdade que $20n^3 + 10n \lg n + 5$ é $O(n^3)$?
3. É verdade que $\frac{1}{2}n^2$ é $O(n)$?
4. Qual a definição formal de $\Theta(f(n))$?

Questão 4 [1 ponto]:

Resolva a recorrência:

$$T(n) = \begin{cases} 2, & \text{se } n = 2 \\ 2T(n/2) + n, & \text{se } n = 2^k, \text{ para } k > 1. \end{cases}$$

Questão 5 [1 ponto]:

Calcule a complexidade de tempo do seguinte algoritmo:

```
1: procedure FUNC(matriz An×n, inteiro n)
2:   j ← 1
3:   while j < n do
4:     k ← 2
5:     while k < n do
6:       S ← S + a[j][k] * a[k][j]
7:       k ← k * k
8:     end while
9:     j ← j + 1
10:  end while
11: end procedure
```

Questão 6 [2 pontos]:

Prove que:

1. todo polinômio de grau k , $p(n) = a_k n^k + a_{k-1} n^{k-1} + \dots + a_0$ com $a_k > 0$, pertence a $\Theta(n^k)$.
2. as funções exponenciais a^n têm diferentes ordens de crescimento para diferentes valores de base $a > 1$.
3. as funções logarítmicas $\log_a n$ têm a mesma ordem de crescimento para diferentes valores de base $a > 1$.

Questão 7 [1 ponto]:

Considere o seguinte algoritmo recursivo, cujo argumento n é um inteiro positivo.

```

1: procedure ASTERISCO(n)
2:   if  $n > 0$  then
3:     ASTERISCO( $n - 1$ )
4:     for  $i \leftarrow 1 \rightarrow n$  do
5:       imprima "*"
6:     end for
7:     ASTERISCO( $n - 1$ )
8:   end if
9: end procedure

```

Para um dado valor de n , quantos asteriscos serão impressos em uma chamada de ASTERISCO(n)? Mostre a recorrência.

Questão 8 [2 pontos]:

Sejam $f(n)$ e $g(n)$ funções assintoticamente não negativas. Usando a definição básica da notação Θ , prove que $\max(f(n), g(n)) = \Theta(f(n) + g(n))$.

$\rightarrow$ podemos provar isto mostrando que $\max(f(n), g(n))$ é $O(f(n) + g(n))$ e que $\max(f(n), g(n))$ é $\Omega(f(n) + g(n))$

Para ① $\max(f(n), g(n)) \leq C \cdot (f(n) + g(n))$ isto é verdade pois a soma de dois elementos não negativos é sempre maior ou igual que o máximo entre eles $\therefore \max(f(n), g(n)) \leq O(f(n) + g(n))$ $C=1$

② $\max(f(n), g(n)) \geq C \cdot (f(n) + g(n))$, para $C = \frac{1}{2}$

temos $\max(f(n), g(n)) \geq \frac{f(n) + g(n)}{2}$ isto é verdade pois a metade da soma de dois elementos é sempre menor ou igual ao máximo dos dois, por exemplo a média aritmética de dois elementos sempre pertence ao intervalo entre estes 2 elementos

Portanto $\max(f(n), g(n)) \geq \Omega(f(n) + g(n))$

Então $\max(f(n), g(n)) = \Theta(f(n) + g(n))$

vel) de que se existe algum candidato em $A[1], \dots, d[3]$ com a maioria dos votos.

Conclusão: o algoritmo Decisão retorna corretamente se houve ou não um candidato com a maioria dos votos em $A[1], \dots, d[3]$.

2) O algoritmo X retorna o maior valor de qualquer vetor $A[1], \dots, n$ passado para X juntamente com seu primeiro e último elemento

Prova $\rightarrow$

✓ 200

Base: para um vetor $A[1], \dots, n$, com $n=1$ temos que o início e o fim de A é igual a 1, portanto o if da linha 2 é executado e o algoritmo retorna $A[\text{inicio}]$ ou seja $A[1]$ pois o maior elemento de um vetor unitário é sempre $A[1]$.

Hipótese: o algoritmo X retorna corretamente o maior elemento de qualquer vetor de tamanho k , $\forall k < n$

Passo: Para qualquer vetor A de tamanho $n \geq 2$ o algoritmo X não irá executar o if da linha 2 portanto ele seguirá para linha 5 onde a variável 'meio' irá conter índice da posição central de A e a entre as linhas 6 e 7 haverá mais duas chamadas de X mais desta vez será passado como argumento na linha 6, um subvetor de A , $A'[\text{inicio}, \dots, \text{meio}]$ e na linha 7, um subvetor de A , $A''[\text{meio}+1, \dots, \text{fim}]$ ou seja dois subvetores de tamanho $k = n/2 < n$ e pela hipótese temos que X nas linhas 6 e 7 retornará corretamente o maior

elementos de cada um dos subvetores possíveis nas linhas 6 e 7 respectivamente para as variáveis a e b onde serão comparados a linha 8 e a partir daí o algoritmo retornará o maior entre eles.

Conclusão: Portanto o algoritmo X retorna corretamente o maior elemento de qualquer vetor $A[1, \dots, n]$. $\triangleright$

3) 1- Dada uma função $f(n)$, e uma função $g(n)$ para que $g(n)$ seja $O(f(n))$ temos que $\{g(n) \mid \exists c \text{ e } n_0, \text{ constantes positivas tais que } \forall n \geq n_0, g(n) \leq c \cdot f(n)\}$

$$2- \lim_{n \rightarrow \infty} \frac{20n^3 + 50n \lg n + 5}{n^3} = \lim_{n \rightarrow \infty} \frac{20n^3}{n^3} + \lim_{n \rightarrow \infty} \frac{50n \lg n}{n^3} + \lim_{n \rightarrow \infty} \frac{5}{n^3}$$

$= 20 \neq 0 \neq \infty \therefore 20n^3 + 50n \lg n + 5$ é $\Theta(n^3)$ o que implica que $20n^3 + 50n \lg n + 5$ é $O(n^3)$. Portanto, é verdade. $\checkmark$

$$3- \lim_{n \rightarrow \infty} \frac{\frac{1}{2}n^2}{n} = \lim_{n \rightarrow \infty} \frac{1}{2}n = \infty \therefore \frac{1}{2}n^2 \in \Omega(n) \quad (0,75)$$

Portanto a alegação é falsa. $\checkmark$ ~~(0,75)~~

4- Dada uma função $f(n)$, e uma função $g(n)$ para que $g(n)$ seja $O(f(n)) \rightarrow \{g(n) \mid \exists c \text{ e } n_0, \text{ constantes positivas, tais que } c \cdot f(n) \leq g(n), \forall n \geq n_0\}$

~~(1)~~

4) usando o teorema mestres temos:

$$a=2, b=2, K=1$$

$$\frac{\lg a}{\lg b} = \frac{\lg 2}{\lg 2} = 1 \therefore$$

$$T(n) \in O(n \lg n)$$

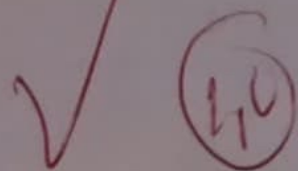
$$T(n) = 2T(n/2) + n$$

$$= 2(2T(n/4) + \frac{n}{2}) + n$$

$$= 2^2(2T(n/8) + \frac{n}{4}) + \frac{n}{2} + n$$

$$\vdots$$
$$= 2^K(T(n/2^K) + \frac{n}{K}) + \frac{n}{K-1} + \dots + n$$

$$2^{K+1} \rightarrow b \lg n = K+1 \rightarrow K = \lg n - 1$$



1- pode ser provado utilizando o limite $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)}$
 onde $f(n) \approx p(n)$ e $g(n) = n^k$

$\rightarrow \lim_{n \rightarrow \infty} \frac{2n^k + 2n^{k-1} + \dots + 2n}{n^k} = \lim_{n \rightarrow \infty} 2k = 2k < \infty$

$\therefore p(n) \in \mathcal{O}(n^k)$

2- pode ser prova utilizando a mesma regra de L'Hôpital

Suponha $a < b \rightarrow \lim_{n \rightarrow \infty} \frac{a^n}{b^n}$ como $b^n > a^n$

$\rightarrow \lim_{n \rightarrow \infty} \frac{a^n}{b^n} = 0$ pois por b ser maior a função b^n cresce muito mais rápido. Portanto $a^n \in \mathcal{O}(b^n)$

agora suponha $a > b \rightarrow \lim_{n \rightarrow \infty} \frac{a^n}{b^n} = \infty$ pois do modo

análogo ao primeiro exemplo a^n cresce muito mais rápido do que b^n . Portanto neste caso $a^n \notin \mathcal{O}(b^n)$

3- isso se deve devido a propriedade de de troca de base por logaritmos onde $\log_k$ pode ser escrito como um $\log$ de base diferente sem alterar seu resultado final.

$\log_a k = \frac{\log_b k}{\log_b a}$ ou seja possuem crescimento iguais

é o limite